Process synchronization

Shared-state races motivate critical-section safety, progress, and bounded waiting. Hardware atomics, mutexes, semaphores, monitors, and classical coordination problems show how these guarantees depend on ordering and resource protocols.

Prerequisites. Process and thread states, Shared memory, Interleaving and atomic operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Learning objectives

- Reason about mutual exclusion, progress, and bounded waiting
- Use semaphores for ordering and backpressure
- Identify deadlock, starvation, priority inversion, and lost wake-ups

Central question. How can arbitrary interleavings preserve shared state while allowing progress and bounded waiting?

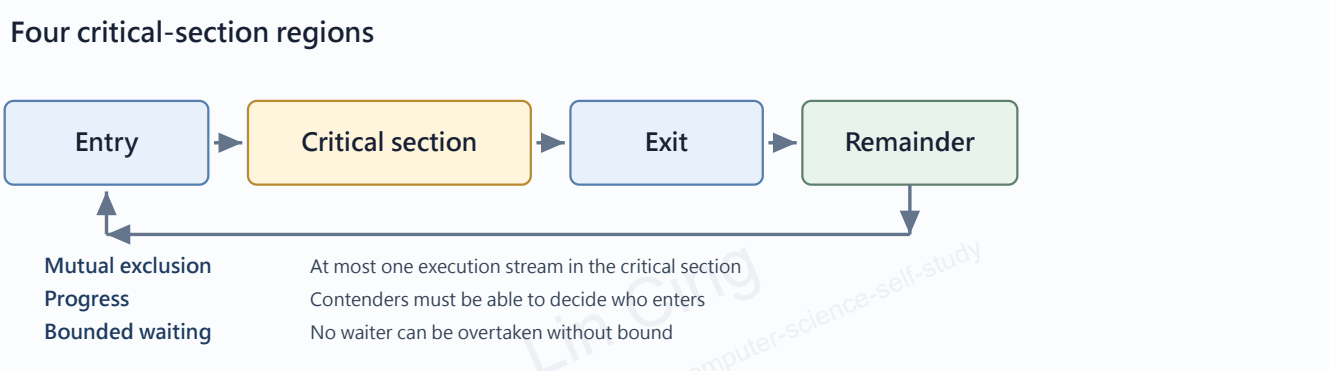
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.1. Background

- A race condition makes correctness depend on uncontrolled event ordering. Shared mutable state requires a protocol for conflicting accesses. A language-level data race has a more specific definition and may make program behavior undefined.
- Atomic operations provide indivisible state changes with specified ordering. Libraries build locks and condition waits from such mechanisms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.1. Background (continued)



Original vector diagram for this course.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.1. Model, ordering, and boundary cases

Model assumptions. Multiple execution contexts access shared mutable state. Atomicity and visibility are defined by the architecture and language model.

Invariant and ordering. Intermediate invalid states remain inaccessible to observers protected by the protocol. Publicly observable states satisfy the shared invariant.

Counterexample or boundary. Individually atomic changes to a list's next pointer and size do not automatically make the combined list update atomic.

Worked check. In an abstract load-add-store trace, two writers read balance 100, each adds 50, and each stores 150. The required result was 200. The rerunnable example models this interleaving without relying on an actual undefined-language race.

Reasoning prompt. Why cannot one atomic instruction always protect an invariant spanning several fields?

5.1 Race invariants and the complete counter trace

Producer executes R1=counter; R1=R1+1; counter=R1. Consumer executes R2=counter; R2=R2−1; counter=R2. The net serial effect is zero, but interleavings can leave 4,5,6 from initial 5. Treating counter++ as one step hides the conflict.

Event	Producer	Consumer	Counter	Observation
T0	R1=5		5	Old value loaded
T1	R1=6		5	Only register changed
T2		R2=5	5	Same old value
T3		R2=4	5	Only register changed
T4	counter=6		6	Producer stores
T5		counter=4	4	Consumer overwrites

5.1 Race invariants and the complete counter trace (continued)

Swapping T4/T5 leaves 6; the intended result is 5. Mutual exclusion covers only agreed regions. Visibility, reordering, non-atomic widths, and failure retries need the same synchronization contract.

Identify all shared mutable locations, state cross-field invariants such as $0 \leq size \leq capacity$ or readers=0 or writer=0, identify operations temporarily breaking them, and define which observers must be excluded plus acquire/release visibility. Atomic stores to a list's next and size do not make their combined update indivisible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.2. The critical-section problem

- Mutual exclusion allows at most one participant in a critical section. Progress prevents indefinite postponement of entry selection when the section is free and contenders exist. Bounded waiting bounds how many entries by others may overtake a participant after its request.
- These are distinct safety, liveness, and fairness requirements. A mutually exclusive protocol can deadlock, and a progressing system can starve one participant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.2. Model, ordering, and boundary cases

Model assumptions. Participants interleave at arbitrary relative speeds within the model. A participant that enters must eventually execute its exit for the usual progress argument.

Invariant and ordering. Entry preserves mutual exclusion, and the protocol cannot let noncontending remainder-section activity prevent a contender from entering.

Counterexample or boundary. Repeatedly selecting favored contenders can preserve system-wide progress while one waiting participant is overtaken without bound.

Worked check. Counter starts at 0. P1 and P2 each load, add 1, and store. P1.load=0, P2.load=0, P1.store=1, P2.store=1 leaves 1 after two increments. Protecting each full read-modify-write with the same mutex produces 2.

Reasoning prompt. Which counterexample refutes each requirement, and which requirements cannot imply the others?

5.2 Safety, progress, and bounded waiting

Each of n processes P0…P(n−1) has entry, critical, exit, and remainder sections.

Requirement	Definition	Failure
Mutual exclusion	While Pi is in its critical section, no other participant is in its critical section	Concurrent writers break one invariant
Progress	If the section is empty and contenders exist, only non-remainder contenders participate in selection, and selection cannot be postponed indefinitely	No contender is selected
Bounded waiting	Every requester has a finite bound on other entries allowed before its admission	Global progress while one requester starves

An unfair exclusive lock can repeatedly admit high-priority arrivals without bounding a lower-priority wait. Progress with exclusion still permits indefinite overtaking. Conversely, admitting everyone can be fast yet violate exclusion. Safety must cover every reachable interleaving; progress and fairness need separate arguments.

5.2 Safety, progress, and bounded waiting (continued)

For enter_i/leave_i, prove exclusion at arbitrary relative speeds; finite-step admission progress when free with contenders; a finite overtaking bound per requester; and the atomicity, ordering, and failure recovery of entry/exit themselves. A finite sample of schedules can find counterexamples, not prove arbitrary n and speeds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.3. Peterson's solution

- For participant i and the other participant j, set flag[i]=true, set turn=j, wait while flag[j] and turn==j, enter the critical section, then set flag[i]=false. Flags announce interest and turn resolves contention.
- With atomic sequentially consistent reads and writes, the two-participant protocol satisfies the classical requirements. Ordinary unsynchronized variables in a modern language do not automatically meet those assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.3. Model, ordering, and boundary cases

Model assumptions. There are exactly two participants. Shared flag and turn accesses are atomic and sequentially consistent, and participants eventually continue executing.

Invariant and ordering. If both flags remain true, the single value of turn excludes one participant. On reentry the winner sets turn to its opponent; a waiting opponent cannot be overtaken repeatedly.

Counterexample or boundary. Compiler or hardware reordering and visibility differences invalidate the sequential-consistency argument unless suitable language atomics and ordering are used.

Worked check. Suppose both could be in the section. Both flags must be true. P0's successful test requires turn not equal to 1, while P1's requires turn not equal to 0 during unresolved contention. Considering the last turn write leaves its writer waiting, contradicting simultaneous entry. With only one flag true, that participant proceeds; with both true, one turn value permits one. The waiting participant is bypassed at most once after the other's relevant exit and reentry under these assumptions.

5.3. Model, ordering, and boundary cases (continued)

Reasoning prompt. Which steps of the proof fail when writes and reads no longer have one sequentially consistent order?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.3 Peterson's complete protocol

There are exactly two processes, j=1−i, atomic flags initially false, and shared turn.

```
P_i:
  flag[i] = true
  turn = j
  while (flag[j] && turn == j) { wait }
  critical_section_i
  flag[i] = false
  remainder_i
```

Flag means competing. Turn resolves simultaneous contention, not permanent lock ownership. The slogan that the last turn writer always loses omits required contention premises.

5.3 Peterson's complete protocol (continued)

Assume atomic sequentially consistent reads/writes. Suppose the two current critical-section attempts overlap. Let W0 be P0's turn=1 and W1 P1's turn=0. If W0<W1, P0's flag0=true preceded W0 and W1. Before P0 leaves, flag0 remains true; neither participant writes turn again inside this attempt. P1 therefore faces flag0=true and turn=0 when it would need to pass, making its while condition true, a contradiction. If W1<W0, symmetry blocks P0. Thus overlapping critical sections are impossible. This compares the current overlapping attempts' final turn writes, not arbitrary past iterations.

For progress, one contender sees the other's false flag and enters. With both competing, turn selects one side whose while is not blocked by turn. A participant already inside eventually clears its flag on exit. For bounded waiting, blocked Pi has flag[j]=true and turn=j. Pj does not change turn in its critical section. After leaving it clears flag; any new attempt first writes turn=i, giving Pi priority. Pi can be overtaken by Pj at most once, assuming the threads continue taking steps and critical sections terminate.

5.3 Peterson's complete protocol (continued)

The proof needs shared order/visibility. Use corresponding atomics with memory_order_seq_cst, or a proved equivalent fencing protocol. Renaming ordinary variables flag/turn supplies no such proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.4. Synchronization hardware

- Test-and-set and compare-and-swap atomically test and change a value. A spinlock repeatedly tries or observes a lock; it is appropriate only when the owner can make progress and the expected wait justifies CPU use.
- Spinning consumes execution resources. High contention or a long critical section can make blocking preferable. Kernel spin protocols also constrain preemption and interrupt contexts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.4. Model, ordering, and boundary cases

Model assumptions. The atomic primitive follows its specified ordering and the lock owner can eventually run.

Invariant and ordering. Only a successful acquisition owns the lock. Release and subsequent acquire provide the required publication of protected updates.

Counterexample or boundary. On one CPU, an interrupt handler that spins on a lock held by the interrupted code prevents that owner from resuming. A preempted owner can likewise suffer priority-related starvation if the spinner never yields. Calling this a nonpreemptive scheduling example would be inconsistent.

Worked check. Two cores attempt CAS from zero to one. Only one succeeds; the other uses bounded backoff or a defined wait strategy. Release stores the unlocked state with suitable ordering.

Reasoning prompt. When is spinning cheaper than sleep and wake, and when does it consume a CPU needed by the owner?

5.4 Atomic hardware and spinning

Test-and-set atomically returns old L and sets true. CAS(&L,0,1) sets 1 only when L was 0 and returns the old value whether it succeeds or not.

```
acquire(L):
  repeat old = test_and_set(L)
  until old == false
release(L):
  L = false
```

One acquire sees false and owns the lock; others see true and retry. Owner-only release with proper publication preserves exclusion. Basic test-and-set does not bound waiting. Queueing, such as a circular waiting array with explicit successor handoff, adds fairness.

Spinning avoids sleep/wakeup and context costs for short sections. Long sections, one core, or a spinner competing with its owner can waste CPU. An interrupt handler spinning on a lock held by the interrupted task can prevent that owner resuming, producing livelock/deadlock. Interrupt contexts need controlled non-sleeping primitives and suitable interrupt/spin rules.

5.4 Atomic hardware and spinning (continued)

Instruction indivisibility does not alone provide language-level acquire/release. Later owners must see prior protected writes; writes before release must remain before publication; compilers must not move critical accesses outside the lock. Use atomic acquire/release semantics or corresponding fences. CAS alone does not solve ABA, reclamation, or lock-free progress.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.5. Mutex locks

- A mutex defines acquisition, ownership, and release. A blocking mutex sleeps when it cannot acquire, reducing CPU waste during long waits.
- A global lock order can eliminate circular waiting among those locks. Error and cancellation paths must also release resources and restore invariants.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.5. Model, ordering, and boundary cases

Model assumptions. Each successful acquisition has an authorized release. Waiters can be awakened under the API's policy.

Invariant and ordering. Protected updates finish consistently before release. All conflicting accesses participate in the same locking or equivalent synchronization rule.

Counterexample or boundary. One thread holds A and waits for B while another holds B and waits for A. Each currently holds one lock, yet their requested acquisitions form a cycle.

Worked check. Require every thread to acquire A before B. Then a cycle would require both increasing and decreasing order, which is impossible. Reverse-order release is conventional, though the absence of acquisition cycles follows from acquisition order. A timeout stress run can find failures but cannot prove all schedules safe.

Reasoning prompt. How do exceptions and cancellation retain the release guarantee?

5.5 Mutex ownership and ordering proof

Every participating critical section uses acquire(m), protected work, release(m). Acquisition/release transitions are atomic. A mutex usually requires its owner to release it; a semaphore need not enforce ownership. Error, cancellation, and exception paths depend on that distinction. Blocking mutex contenders wait; spinners continue consuming CPU and require short sections with a runnable owner.

Assign a strict global lock order, ord(A)=1, ord(B)=2. Acquire a new lock only above every held lock. Any resource-wait cycle would require ord(A)<…<ord(A), a contradiction. In particular holding B while acquiring A violates ord(B)<ord(A). Callbacks, condition waits, upgrades, exceptions, and cancellation must obey the same rule. Sort dynamic locks by a stable address or unique ID, not transaction argument order such as from/to accounts.

5.6. Semaphores

- A counting semaphore represents permits. In one blocking implementation, wait decrements an internal signed value and blocks if it becomes negative; signal increments it and wakes a waiter when required. Other implementations keep a nonnegative permit count and a separate wait queue.
- A binary semaphore can represent an event or support an exclusion protocol, but it does not inherently have mutex ownership. Extra signals can corrupt a resource-accounting protocol.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.6. Model, ordering, and boundary cases

Model assumptions. Wait and signal, including their queue transitions, are atomic under the semaphore abstraction.

Invariant and ordering. For a fixed pool of C resources, available resources plus outstanding allocations equal C. A signed implementation value may be negative to count waiters; that value is not a negative number of physical resources. Event semaphores need not be bounded by their initial value.

Counterexample or boundary. A producer that holds the buffer mutex while waiting for an empty slot can block the consumer that must create that slot.

Worked check. A three-slot buffer starts with empty=3 and full=0. After three completed productions and no consumption, full=3 and no empty permit exists. A fourth producer blocks before taking the buffer mutex.

Reasoning prompt. Which error paths become possible when a semaphore does not enforce ownership of permits?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.6 Permit abstraction and signed implementation

A resource semaphore counts permits; a binary abstraction has 0 or 1 free permits. A signed blocking implementation also encodes waiters. Counter and queue actions must be atomic with blocking/wakeup.

```
busy_wait(S): while (S <= 0) {} ; S--
busy_signal(S): S++

wait(S):
  S--
  if S < 0:
    enqueue current on S.queue
    block current
signal(S):
  S++
  if S <= 0:
    dequeue one waiter
    wake it
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.6 Permit abstraction and signed implementation (continued)

The busy-wait notation requires indivisible successful test/decrement, not two racing ordinary operations. Negative signed state represents waiters, not negative resources. At quiescent boundaries, free=max(S,0), queued=max(−S,0). A handed-off permit is allocated even if its recipient has not received CPU; newcomers cannot take it. Resource semaphores preserve free+allocated=capacity. Event semaphores need not have fixed capacity.

For capacity 1: T0 wait 1→0 obtains the permit; T1 wait 0→−1 queues/sleeps. T0 signal−1→0 wakes T1 because the incremented value≤0 and hands it the permit. T1 resumes after its original wait without decrementing again. T1 signal0→1 leaves a free permit because nobody waits. Waking only on0→1 strands T1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.6 Permit abstraction and signed implementation (continued)

For capacity 3, empty=3, full=0, mutex=1:

Completed operation	Empty	Full	Buffer
Initial	3	0	[]
Put A	2	1	[A]
Put B	1	2	[A, B]
Put C	0	3	[A, B, C]
Get A	1	2	[B, C]
Put D	0	3	[B, C, D]

Producer uses wait(empty), wait(mutex), update, signal(mutex), signal(full). Consumer first waits full. Holding mutex while awaiting capacity can prevent the consumer releasing capacity.

5.6 Permit abstraction and signed implementation (continued)

If P0 takes S then Q and P1 Q then S, each initially1, P0 can hold S and P1 Q before waiting forever for the other. Starvation instead leaves some work progressing while a waiter is indefinitely skipped; LIFO arrivals can do that. FIFO removes one starvation pattern but not incorrect pairing.

For priority inversion, low-priority L owns H's needed resource while medium M preempts L. Inheritance temporarily raises L to H's priority until the section completes, then restores it. The historical Mars Pathfinder VxWorks incident illustrates this mechanism.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.7. Classical synchronization problems

- A bounded buffer uses empty and full for capacity and a mutex for its representation. Producers wait for empty before the mutex, publish an item under the mutex, release the mutex, and signal full. Consumers symmetrically wait for full before the mutex and signal empty after removal.
- Readers-writers permits simultaneous readers and exclusive writers, but fairness policy matters. Reader preference can starve a writer. Dining philosophers demonstrates circular waiting during incremental resource acquisition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.7. Model, ordering, and boundary cases

Model assumptions. Acquisitions are not automatically all-or-nothing. Scheduling and admission assumptions must be stated separately from deadlock prevention.

Invariant and ordering. Buffer occupancy matches published item and free-slot accounting, including in-flight reservations. A philosopher eats only while holding both required forks and releases them afterward.

Counterexample or boundary. If all five philosophers hold their left fork and wait for their right, none can proceed.

Worked check. Admit at most four philosophers to the five-fork table. The chain of occupied left forks has a boundary next to a free fork, so some admitted philosopher can acquire its second fork and finish, assuming it is scheduled. This breaks the all-five cycle but does not itself guarantee fair admission or starvation freedom.

Reasoning prompt. Why does deadlock freedom fail to imply starvation freedom?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.7 Classical coordination protocols

Empty prevents producing into a full buffer, full prevents consuming an empty one, and mutex protects compound slot/head/tail/size updates. Capacity waits occur outside mutex. Otherwise producer waits for consumer capacity while consumer waits for producer's mutex. The occupancy safety condition is $0 \leq \text{items} \leq n$. General capacity accounting is $E+F+P+C=n$. E and F are the nonnegative, unreserved free permits of empty and full. P counts producers already granted an empty permit that have not yet signaled full; C counts consumers already granted a full permit that have not yet signaled empty. A permit handed directly to an awakened waiter counts as its P or C reservation even before it runs.

Every atomic protocol transition preserves the sum. Successful producer wait moves E to P; producer signal(full) moves P to F, or directly to a consumer's C on handoff. Consumer wait moves F to C; signal(empty) moves C to E or directly to another producer's P. Writing or removing the item does not change this capacity classification. With capacity 3 and an empty buffer, (E,F,P,C)=(3,0,0,0). After only one successful wait(empty), it is (2,0,1,0), so E+F=2, even before insertion. E+F=n holds at quiescent boundaries with no reservations, P=C=0.

5.7 Classical coordination protocols (continued)

E/F are not signed semaphore raw counters. A negative raw value also encodes queued waiters and cannot be substituted into permit conservation. Raw zero can coexist with an outstanding handoff grant. Modify the structure under mutex, publish full only after insertion, and publish empty only after removal. These rules preserve occupancy safety; F need not equal the number of stored items during an operation.

Reader-preference allows new readers while writers wait, but not while a writer owns access; writers may starve. Writer-preference stops new readers after a writer starts waiting; readers may starve. Both allow concurrent readers when permitted. In the first protocol, read_count is protected by mutex; first reader acquires rw_mutex, last releases it. A lock protects its actual state, not everything its name suggests.

5.7 Classical coordination protocols (continued)

Five philosophers each take chopstick[i], then chopstick[(i+1)%5], eat, release right then left. All can hold left and wait right, producing a five-edge cycle. Alternatives limit simultaneous contenders to four, acquire both atomically when available, or use opposite acquisition orders for odd/even philosophers. They exclude this deadlock pattern without automatically proving starvation freedom; waiter selection still matters.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.8 Wait for a predicate, not a notification ticket

Use Mesa semantics, one mutex `m`, condition variable `not_empty`, and an initially empty shared queue `q`. P inserts A once. C1 and C2 each call `get()` once. Every queue mutation and `q.empty()` check holds the same mutex.

```
get():                          put_A():
  lock(m)                         lock(m)
  while q.empty():                q.push_back(A)
    wait(not_empty, m)            signal(not_empty)
  x = q.pop_front()               unlock(m)
  unlock(m)
  return x
```

Wait combines lock release and wait registration atomically relative to another thread acquiring m and notifying; successful return reacquires m. Signal does not transfer the mutex or reserve A for the waiter. With no waiter, no permit is accumulated.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.8 Notification does not determine the next lock owner

Time flows down; P inserts one A; no spurious wakeups in this trace.

	Consumer C1	Producer P	Consumer C2	mutex / queue
T0	wait: unlock + enqueue			m=—, q=[]
T1		lock; enqueue A		m=P, q=[A]
T2	notified; no lock yet	signal C1; keep m		m=P, q=[A]
T3		unlock →	lock; take A; unlock	m=—, q=[]
T4	relock; q is empty			m=C1, q=[]
T5	while → wait again			m=—, q=[]

T3 orders P's unlock, C2's lock and removal of A, then C2's unlock. The mutex is not assumed FIFO. C1's T4 wait return owns m, but the predicate is false again. No spurious wakeup occurs in this counterexample: `if` would allow an empty pop; `while` sends C1 back to waiting at T5.

5.8 Track three distinct forms of state

CV holds condition waiters. E contains notified waiters awaiting m, **not a kernel ready queue**. Notification changes eligibility, not ownership or predicate truth. States follow each event.

Event	m owner	q	CV	E
T0 C1 waits	None	`[]`	`{C1}`	`{}`
T1 P inserts A	P	`[A]`	`{C1}`	`{}`
T2 P signals C1	P	`[A]`	`{}`	`{C1}`
T3 C2 removes A and unlocks	None	`[]`	`{}`	`{C1}`
T4 C1 reacquires	C1	`[]`	`{}`	`{}`
T5 C1 waits again	None	`[]`	`{C1}`	`{}`

5.8 Lost wakeup comes from a gap before waiting

Broken: separate unlock and wait

1. C1: sees empty, unlocks; not yet queued
2. P: inserts A, signals; nobody waiting
3. C1: queues and sleeps too late; q=[A]

Data exists, but C1 sleeps; signals do not accumulate.

Correct: atomic wait contract

1. C1: sees empty; wait unlocks + registers
2. P: locks, inserts A, signals, unlocks
3. C1: relocks, rechecks, then takes A

If P arrives first, C1 sees A and skips wait.

The left side is a broken abstract wait implementation: unlock first, register and block later. It is **not legal `pthread_cond_wait` usage**. P inserts and signals in the gap while CV is empty; C1 then sleeps despite queued data. Atomic wait excludes that gap. If P completes first instead, C1 sees a nonempty queue under m and skips waiting.

5.8 Prove safety and liveness separately

Safety. Every pop checks `!q.empty()` while holding m. No other thread can acquire m and change q between this check and removal. Wait return guarantees reacquisition, not predicate truth.

Liveness. The while loop prevents an invalid removal, but does not promise data for every consumer. There is only one A; once C2 consumes it, C1 may legitimately continue waiting. Scheduler fairness, mutex fairness, and future arrivals require separate assumptions.

If broadcast wakes two consumers for one A, the first owner takes A; the other returns to the while loop. Waking more threads creates neither data nor a bounded-waiting guarantee.

5.8 Handoff contracts: Hoare, Mesa, and signal-and-exit

The trace uses Mesa semantics: the signaler continues, and the waiter reacquires later. Hoare's handoff-time condition guarantee must not be imported into that model.

Handoff	Lock/execution after signal	Waiter reasoning
Hoare signal-and-wait	Immediate handoff; signaler waits	May use the condition established at handoff under that language's contract
Mesa signal-and-continue	Signaler retains the lock; waiter reacquires later	Recheck the predicate
Concurrent Pascal signal-and-exit	Signaler leaves, then hands off	Language-defined

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.9. Synchronization examples

- The book's Solaris, Windows, Linux, Java, and Pthreads cases are versioned illustrations. Similar names do not establish identical ownership, fairness, or cancellation behavior.
- Java synchronized uses an object's monitor. Its wait and notification operations require ownership of the corresponding monitor. Other APIs have their own contracts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.9. Model, ordering, and boundary cases

Model assumptions. Each API's semantics and permitted calling contexts are explicit.

Invariant and ordering. Check ownership, wake-up behavior, interrupt context, priority rules, and acquisition order as one protocol.

Counterexample or boundary. Taking an ordinary sleeping mutex in a non-sleepable interrupt context can deadlock or trigger a kernel failure.

Worked check. Windows 7 dispatcher mutex M starts signaled. T1 acquires it, making it nonsignaled. T2 waits on its queue. T1 releases, making M signaled; the kernel selects one waiter, T2 moves waiting→ready, then acquires M, which becomes nonsignaled again.

Reasoning prompt. Which guarantee would require a version-specific specification before using it in a current implementation?

5.9 Historical synchronization environments

Historical case	Mechanism	Boundary
Windows 7 kernel	SMP spinlocks for short global-resource sections	Holder not preempted
Windows 7 user services	Dispatcher mutex, semaphore, event, timer	Signaled/nonsignaled states; mutex grants one owner
Linux 2.6 onward textbook snapshot	Preemptible kernel, atomic_t, mutex, spinlock, semaphore	Short SMP spin regions; preempt_count>0 prevents safe preemption
Solaris textbook snapshot	Adaptive mutex, RW lock, turnstile	Spin while holder runs, otherwise sleep; turnstile supports inheritance
Pthreads/POSIX	Mutex, CV, RW lock; POSIX SEM semaphores	API determines waits, wakeups, errors

5.9 Historical synchronization environments (continued)

The textbook Windows critical-section object first spins in user mode and uses kernel assistance under prolonged contention. The cited uniprocessor Linux case replaces ordinary SMP-style spinning with preemption control. Solaris selects spin/sleep according to holder execution. These are versioned cases, not all-current-system claims.

Before substitution, identify user/kernel/interrupt context, scheduling entity, ownership and process-sharing support, fairness/inheritance/cancellation/wakeup order, and versioned memory ordering. Similar names do not establish equivalence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.10. Alternative approaches

- Transactional memory commits a region atomically or aborts it. Retrying requires that effects are replayable and that external side effects obey the transaction contract.
- Message passing and immutable data can reduce shared mutable state. Lock-free structures still share mutable state but provide a different progress guarantee. All designs retain costs in conflict handling, scheduling, or reclamation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.10. Model, ordering, and boundary cases

Model assumptions. Conflict detection, retries, progress, and memory reclamation have explicit rules.

Invariant and ordering. Successful commit exposes the specified atomic result. Abort leaves no visible partial transaction effects under the model.

Counterexample or boundary. A lock-free queue that immediately frees a node still referenced by another thread can cause use-after-free despite atomic pointer updates.

Worked check. Start A=100, B=200. T1 computes A−=25, B+=25. A pre-commit read-set conflict aborts, leaving 100/200. A successful retry atomically publishes 75/225. The abort and commit cannot each leave a partial debit or credit; an external payment effect also cannot be blindly repeated.

Reasoning prompt. Why does a low conflict rate alone fail to establish efficient transactions?

5.10 Alternatives and their remaining boundaries

A memory transaction makes a sequence of shared accesses all-or-nothing. Commit publishes it; abort rolls it back, excluding partial observation. For atomic{x=x−25; y=y+25}, conflicting read/write sets can force one transaction to retry. Aborting can avoid lock-order deadlock but still permits starvation and must handle I/O, irreversible effects, and retries. STM instruments accesses through compiler/runtime; HTM uses coherent-cache mechanisms. Capacity and failure paths differ.

OpenMP critical regions of the same name exclude one another; distinct names identify distinct regions. The programmer still identifies shared conflicts. Immutable functional values avoid in-place writes and permit sharing old values, but files, networks, mutable databases, and mailboxes remain shared resources. Message copying/ownership transfer reduces shared mutation without eliminating capacity, ordering, backpressure, or retry coordination.

5.11. Summary

- Synchronization combines state invariants with the required progress and fairness guarantees. Higher-level operations must connect predicates, publication, waiting, and wake-up correctly.
- Final values alone can miss illegal intermediate states and timing failures. Model interleavings and check the observable invariant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.11. Model, ordering, and boundary cases

Model assumptions. Analysis includes every access and failure path. Lock-free algorithms also specify reclamation and their precise progress property.

Invariant and ordering. Operations preserve the abstract state, and the required progress holds under stated scheduling assumptions.

Counterexample or boundary. Releasing a lock only on normal return leaves later operations blocked after an exception.

Worked check. Two threads each execute load, increment, store, giving C(6,3)=20 order-preserving interleavings. Loading 0 in both before either stores leaves 1; other interleavings give 1 or 2. With the same mutex protecting the whole RMW, only P1-then-P2 or P2-then-P1 remains, both giving 2. This finite model does not replace the general lock proof.

Reasoning prompt. How can cancellation and exception transitions be included in a synchronization check?

5.11 Choosing a synchronization primitive

Need	Required model	Check
One critical section	Length, sleepability, ownership	Paired acquire/release
Fixed resources	Capacity, initial permits, authorized signalers	No excess signal
Predicate wait	Variables, notification, recheck	One protecting mutex
Multiple locks	Global order, dynamic key, cancel paths	Every acquisition follows order
Readers/writers	Preference/fairness rules	Reader/writer starvation

Trace modes, owned resources, waiters, and predicates after selection. Typical successful tests do not exclude rare schedules. Supply general proofs and rerunnable counterexamples separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter conclusions

- Critical-section requirements separate safety, progress, and fairness
- Semaphore and monitor errors often arise from predicates and wait order
- Classical proofs require their stated memory-model assumptions

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 9th edition, Wiley, 2013. Historical system examples use the versions identified in this chapter.

POSIX.1-2024 condition-wait ownership, atomic release, and reacquisition contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study